# Building XR Safety-Training Examples with an AI Coding Assistant: A Three-Week Experience Report on Verification, Hardware Gaps, and Honest Reporting

**Jeremy Callinan**

University of Pittsburgh at Bradford · Manuscript prepared November 8, 2026 · Repository state described: commit `f4e5899` of github.com/jpcst52/ALL-VR-MR-XR-SDKs

## Abstract

Immersive virtual and augmented reality (XR) is promoted for safety training because the real tasks are hazardous, yet small campuses rarely have the staff to build such content, and the evidence that XR improves learning is mixed. This experience report describes a three-week effort (October 17 to November 8, 2026) in which one educator, working with an AI coding assistant (Claude, used through Claude Code), built a public repository of XR training examples for an advisory-board demonstration. The work grew to seventeen hands-on browser scenarios, multi-user WebXR labs, four Unity projects and a lobby hub for SteamVR and the HTC Vive, and the tooling around them. The report draws on the repository's history (63 commits), its status and verification records, and an incident log, not on participant data. It makes three contributions. First, it describes the working arrangement: a project-instructions file, a task dashboard, parallel sub-agents with ownership rules, and repeated review rounds. Second, it classifies the defects and process failures that surfaced (editor-versus-player divergence, the emulator-to-hardware gap, undocumented platform limits, overreaching research claims, and diverging parallel work) and the controls that caught them. Third, it proposes an evidence ladder for stating what has and has not been verified. At the time of writing no example has been tested in a headset and no learning outcome has been measured. The report treats that gap as a central finding about AI-accelerated development, not as a footnote.

**Keywords:** virtual reality; WebXR; Unity; safety training; AI-assisted software development; experience report; verification

# 1. Introduction

Training for hazardous work is a natural use for immersive technology: the learner can make mistakes that would be dangerous, costly or impossible in a real plant. A meta-analysis of VR, AR and mixed-reality training found it about as effective as commonly accepted methods overall, and concluded that its value lies mainly where danger or cost make traditional training impossible (Kaplan et al., 2021). For an engineering-technology program at a small regional campus, that argument is attractive, but the practical obstacle is not the idea. It is the labour of producing content that runs on the available headsets, looks credible to employers, and can be shown on a fixed date.

This report describes how that obstacle was approached in one case. In autumn 2026 the author, who teaches and builds XR training at the University of Pittsburgh at Bradford, set out to prepare examples for an advisory-board demonstration on November 13, 2026. The audience was to include local employers and faculty from energy, industrial and biology programs. The hardware was Meta Quest headsets running the Quest Browser, an HTC Vive Pro 2 on SteamVR, and desktop computers with RTX-class GPUs. Almost all of the implementation was produced in working sessions with an AI coding assistant, Claude, accessed through the Claude Code command-line and editor tools. The author set direction, made decisions and corrected the assistant. The assistant wrote and ran code, wrote documentation, and coordinated sub-agents.

The report asks three questions.

1. **RQ1.** What did a three-week, AI-assisted build of XR training examples produce, and how was the work organised?
2. **RQ2.** Where did defects, overclaims and process failures arise, and which controls caught them?
3. **RQ3.** What practices follow for educators who build XR training with an AI assistant, especially when the hardware is not at hand?

The report is an experience report, not an evaluation of learning. No participants were involved and no learning data exist. Its value is as a documented case with enough concrete incidents to support transferable practices, and as a candid record of what had and had not been verified on the day of writing.

# 2. Background and related work

## *2.1 Evidence on immersive and safety training*

Reviews of immersive VR in higher education found that learning theory was often not used to guide design and that evaluations measured usability more often than learning outcomes (Radianti et al., 2020). A review of 29 studies reported an advantage for immersive VR in most comparisons, but short interventions, few retention measures and weak assessment methods (Hamilton et al., 2021). For safety training specifically, a systematic review and meta-analysis of 52 articles found VR training outperformed traditional training for knowledge acquisition and retention, while noting that few studies adopted explicit theory or measured long-term retention (Scorgie et al., 2024). A review of 136 studies, organised by Kirkpatrick's levels, found that most measured learning and reaction and very few measured behaviour or results (Stefan et al., 2023). A process-industry case study found VR learning of complex safety tasks statistically equivalent to slide-based video training (Poyade et al., 2021), and a study of aviation-safety training found knowledge and self-efficacy gains, maintained at two weeks, regardless of display type (Buttussi & Chittaro, 2018). *(These summaries rest on abstracts and records; they are cited for context, not for effect sizes.)*

Immersion is not automatically beneficial. In a study of 52 university students, an immersive version of a science lab simulation produced more presence but less learning and higher EEG-measured cognitive load than the same simulation on a desktop (Makransky et al., 2019). In a related experiment a desktop slideshow produced higher posttest scores than an immersive VR lesson, although the VR group reported more interest and motivation (Parong & Mayer, 2018). A recent review of 36 studies reports that VR, which often causes extraneous load, hinders learning, particularly for novices (Poupard et al., 2025). This matters for the case below, because a large share of the effort went into visual and audio realism.

## *2.2 Design frameworks*

Slater (2009) proposes place illusion and plausibility illusion as the two components that produce realistic responses in VR; consequences that respond to the learner's actions target plausibility more than visual fidelity does. Johnson-Glenberg (2018) argues that presence and hand-based manipulation are VR's distinctive educational affordances. Cognitive load theory (Sweller, 1988) and multimedia-learning principles (Mayer & Moreno, 2003) motivate treating extraneous detail as a cost. Two small studies found no difference in learning or presence between hand tracking and controllers (Khundam et al., 2021; Saran, 2025), which suggests that task design may matter more than input device.

## *2.3 WebXR and multi-user learning*

Evidence specific to browser-based XR is thin. One acceptance survey of a WebVR virtual campus reported general acceptance with technical problems and connection quality as

weaknesses (Rocha Estrada et al., 2022). A review of 139 articles on collaborative VR learning found that most systems used monitor-based VR with mouse and keyboard (van der Meer et al., 2023), and a review of 27 studies in engineering education reported benefits for engagement and teamwork alongside barriers of cost and technical problems (Dunmoye et al., 2025).

### *2.4 AI-assisted programming*

A within-subjects study with 24 participants found that an LLM-based code generation tool did not necessarily improve task completion time or success rate, but participants preferred it because it provided a useful starting point, and they struggled to understand, edit and debug generated code (Vaithilingam et al., 2022). A grounded-theory study of 20 participants described interactions as bimodal: in *acceleration* mode the programmer knows what to do next and uses the assistant to get there faster, and in *exploration* mode the programmer is unsure how to proceed and uses the assistant to explore options (Barke et al., 2023). A survey of hallucination in natural language generation describes the tendency of generative models to produce unintended text that degrades performance and fails user expectations (Ji et al., 2023). These studies concern programmers working on conventional tasks. They say little about educators building cross-platform XR content on a deadline, where much of the correctness depends on hardware that is not in the room. That is the setting of this report.

### *2.5 Method*

The report follows the spirit of design-science research, in which an artifact is built and the knowledge gained is stated (Hevner et al., 2004; Peffers et al., 2007), but it does not claim a full design-science evaluation. It is a reflective case study of one project, using the repository as the primary record.

## 3. The project and how the evidence was gathered

### *3.1 What was built*

Table 1 summarises the artifacts. Sizes are computed from the tracked files of the repository at commit `f4e5899` (3,664 files). They exclude vendored libraries, generated site output and the licensed assets that are deliberately not committed: about 20,600 lines of JavaScript, about 29,900 lines of C#, and about 113,000 words of Markdown documentation.

*Table 1. Artifacts produced (state at commit f4e5899, November 8, 2026).*

| Group | Content | Target |
|---|---|---|
| Browser scenarios | A hands-on "Practice Lab" that grew from 12 to 17 scenarios (business, research, industrial, and a biology draft for faculty review), each ending in a "why this matters" reasoning step | Desktop browser, Quest Browser (WebXR) |
| Multi-user labs | A Node relay and shared-board labs (a circuit lab, a site-safety layout, a hydrostatic-test builder) for VR and AR, with an in-headset toolbox, plus a second-generation look ("v2") | Quest Browser, desktop, phones |
| Single-SDK demos | Four browser demos (A-Frame, Three.js, Babylon.js, raw WebXR) and ten SDK or framework examples with READMEs | Browser; native samples not run here |
| Unity projects | A shared kit (rig, teleport and snap turn, hands, signage, lighting, synthesised sound) and four scenarios: hydrostatic test, refinery rounds, warehouse and forklift, resistor bench | OpenXR on SteamVR, HTC Vive Pro 2 |
| Hub | A Unity lobby with four portals and dressed versions of the scenarios using licensed Asset Store packs and Microsoft Rocketbox people (Gonzalez-Franco et al., 2020) | SteamVR; builds on the lab PC only |
| Tooling | An emulated-Quest capture and check suite, a status dashboard, a deck builder, build and pack scripts, a GitHub Pages workflow with a link check | Windows development PC |

Scenario content is general safety awareness, not a procedure or certification, and every README says so. Browser examples had to run offline, with libraries vendored and no network calls, because the demonstration venue's connectivity could not be assumed. Web demos used only CC0 assets. Licensed Unity packs were allowed in local builds but never committed, which later shaped the build tooling (Section 5.4).

## *3.2 The working arrangement*

Four mechanisms organised the collaboration.

- **A project-instructions file.** A `CLAUDE.md` at the repository root, read at the start of every session, recorded the owner's working preferences (complete deliverables, honesty about what is verified, simple over clever, screenshots after visual changes), repository conventions (offline libraries, what must not be committed, brand palette) and standing instructions (compile before handing over Unity code; keep the status dashboard current).
- **A task list and dashboard.** A Markdown task table was rendered by a small local dashboard. Tasks, sub-agents and test results were recorded there as they changed, so that

the next session did not have to rediscover the state.

- **Sub-agents with ownership rules.** For parallel work (for example browser QA rounds), a rules file told each agent to stay inside assigned files, to re-read shared files before editing, never to run `git add`, `commit` or `push`, to report only what it actually ran, and never to claim that anything works on a Vive or Quest. The main session reviewed and committed.
- **Review rounds.** The commit history names successive rounds (round 1 review, round 2 compliance sweep and content review, round 3 detail pass), each ending with test counts and documents refreshed.

### *3.3 Data sources and positionality*

The evidence comes from: the git history (63 commits, October 17 to November 8, 2026); the status, testing and build documents in the repository; a research verification record; the task list; and incident notes kept during development. The author is the educator who directed the work and is also the subject of this report. The assistant produced much of the text of the records it is now cited from, and the first draft of this manuscript. The limits this creates are discussed in Section 7.

## 4. The three weeks

Figure 1 shows commits per day. The history is bursty: a quiet start, a long gap, then two days with 38 commits in total (November 2 and 3, of which 30 on November 3 alone), and a tail of smaller, hardware-oriented work. Authorship in the history cannot separate human from assistant work: 58 of the 63 commits carry the owner's name (including those written in assistant sessions), four come from a second account, and one from a GitHub agent that produced the first slide deck.

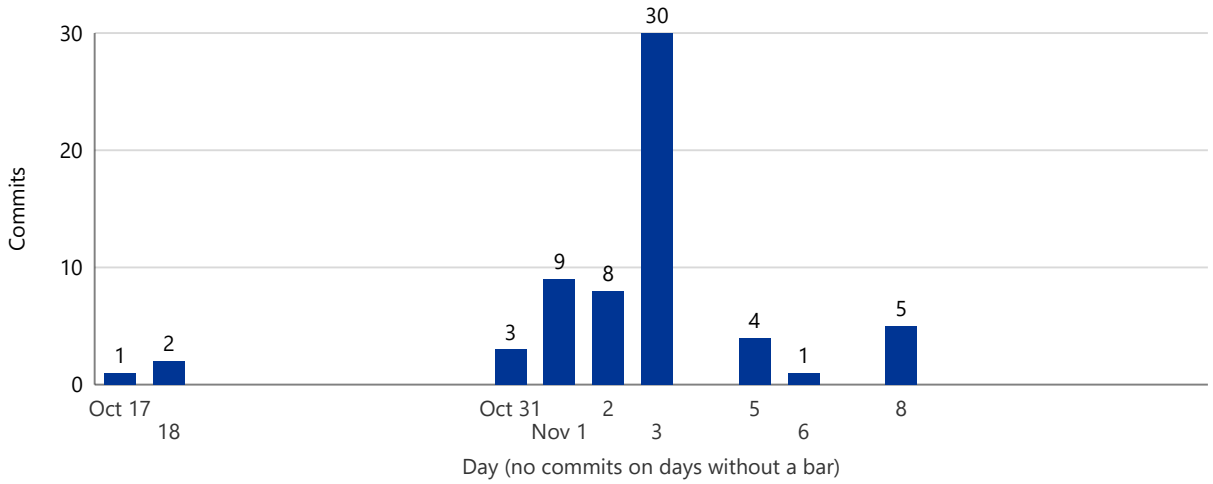



Figure 1. Commits per day, October 17 to November 8, 2026 (n = 63; source: `git log`). The branch discussed in Section 5.7 is not included.

**Phase 0 (October 17–18).** A first set of SDK examples and an overview slide deck.

**Phase 1 (October 31–November 1): breadth.** Multi-user WebXR labs, the hands-on Practice Lab, offline vendoring of Three.js, Babylon.js, A-Frame and fonts, a research bibliography, the shared Unity kit and the first Unity project, the status dashboard, and the hub project. Ten framework examples, twelve scenarios and four Unity projects existed in some form by the end of this phase, in the sense that code and READMEs existed; most had been run only on a desktop.

**Phase 2 (November 2–3): depth and review.** Branding, "baking" Unity scenes into real GameObjects so that they could be inspected and edited, the hub lobby and people, three review rounds, a documentation checker, a GitHub Pages workflow, a demo server with port fallback, and a re-verification of the bibliography. The concentration of 38 commits in two days reflects review rounds that fed fixes back into the code.

**Phase 3 (November 5–8): toward the headset.** Features that only matter in a headset (an in-headset toolbox, Y and B buttons, teleport and snap turn, visible hands, a stereo-safe text shader), a rewrite of sound synthesis, lighting passes, realistic people in the warehouse project, and, on November 8, the first Windows player builds, a lab-test kit, and the hub build with its tooling.

## 5. Findings: incidents, mechanisms and controls

Each finding below follows the same pattern: what happened, why the existing checks did not catch it, and what was changed. Appendix B lists them in one table.

### *5.1 An evidence ladder was needed, and most work sat on its lower rungs*

During the project the owner corrected a statement that a software library "works on the Vive" that had come only from the library's README. The correction became a standing rule: say what was run, what was only emulated, and what is unverified; never claim hardware works unless it was tested. The rule is worth stating as a ladder, because the assistant could climb most rungs alone but not the last:

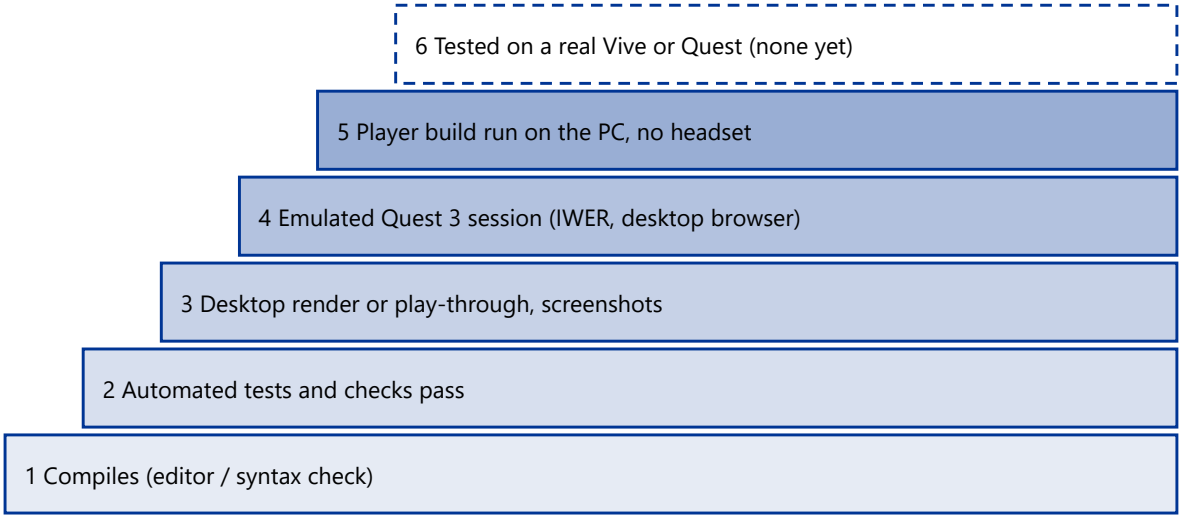



Figure 2. The evidence ladder used to label claims. As of November 8, 2026 no example had reached rung 6.

*Table 2. Highest rung reached by each group on November 8, 2026.*

| Group | Highest rung | Evidence |
|---|---|---|
| Browser scenarios and labs | 4 | Suites recorded in `docs/TESTING.md` (for example Practice Lab 36, relay 64, UI layout 48, multi-user end-to-end 81, demos 12, 26 and 4 of 4 checks); a 41-check toolbox suite in an emulated Quest 3. No headset. |
| Unity projects 01 to 04 | 5 (03 only briefly) | Compile checks and desktop captures for all; Windows players built for all four on November 8; the warehouse player was started windowless for 25 seconds. No headset. |
| Hub | 5 | Player built (about 1 GB), started windowed for 45 seconds, lobby loaded, lobby music generated, no exceptions. No headset; navigation by portals was not exercised. |
| Sound | 2 | Synthesised clips were measured for loudness; they had not been listened to in a scene. |
| Lighting and realism | 3 | Judged on desktop renders; not judged in a headset. |

*Lesson.* A single status word (“done”) hides the ladder. Labelling each claim with its rung made it easy for the owner to see what the lab day had to test, and the lab-test guide was organised around the claims on rungs 4 and 5 that were unverified at rung 6.

## *5.2 Editor and player diverge*

The shared Unity kit had a standing rule to compile after every edit, using a headless compile check. On November 8, the first attempt to build Windows players failed for all four projects. The cause was one line in a runtime script that set `Light.shadowRadius`, a property that exists only in the editor. The editor compile check had passed, because the editor assembly sees editor-only members. The line was removed and the builds succeeded. A second instance appeared in the hub: a sky shader that the code located with `Shader.Find` was stripped from the player, because no scene referenced it, so the lobby silently fell back to a plain sky; the player log showed a warning. A pre-build hook now lists the shader under Always Included Shaders.

The same episode contains a false alarm that is instructive in its own right. A first player run used `-nographics` and still showed the shader warning after the fix. A second run with graphics enabled showed no warning. The earlier log line had been an artefact of the way the check was run. A check that cannot distinguish a fixed defect from an unfixed one is not a check.

*Lesson.* For platforms with an editor, “compiles in the editor” sits well below “a player builds and starts”. The control adopted was to build and run the player before handing anything over, and to treat unexpected log lines as test output.

### *5.3 Some defects cannot appear on the development display*

The task list records a defect in which text appeared in one eye only. The cause was a text shader that lacked the macros required for single-pass instanced stereo rendering. A related kit change replaced a colour shader with a stereo-aware one in the warehouse's awareness shell. A defect of this kind is impossible to see in desktop captures, which have one view. The fix was verified only by compilation and by desktop self-tests that were added for the rig and hands. The task list entry says plainly: nothing tried on a headset.

*Lesson.* Assistant-driven verification is limited to what the assistant can observe. Where a class of defect needs a different display or a human body (stereo, comfort, controller feel), the correct output is a list of things to look for, not a claim that the thing was checked. The lab-test guide contains exactly such a list.

### *5.4 Implicit platform limits surface at integration*

Three constraints that nobody had written down failed in the final days.

- **A deployment image cap.** The GitHub Pages build for the browser examples silently omitted any image above 3 MB from the root README and the browser example guides (700 KB for other guides), and the link check then failed the deployment. A 3.3 MB launcher screenshot triggered it. The images were reduced to 256 colours (3.3 MB to 0.84 MB and 3.0 MB to 0.80 MB, at the same dimensions) and a check script was added to the workflow so that the failure names the file before the build starts.
- **Licensed content and a clean checkout.** The hub's scenes, art packs and people are licensed or large and are git-ignored. A fresh checkout therefore cannot build the hub. A build script that overwrote the “combined” archive with a hub-only build would have put licensed content in a file intended for sharing. The script now checks the hub's prerequisites before starting Unity, keeps the hub out of the combined archive, and a guide and pack/unpack scripts document how to move the local files between PCs.
- **Editor modal states.** A killed Unity editor leaves backup scene data that blocks the next headless run with a recovery prompt; compile errors in the shared kit put every project into Safe Mode. Both are now recorded in the troubleshooting notes.

*Lesson.* Every external system (a hosting service, a licence, a vendor tool) carries limits that are discovered by hitting them. The practice that worked was to convert each discovery into an

automated guard and a sentence in the setup guide, so that the next session, human or assistant, meets the limit in advance.

## *5.5 Research claims drift beyond their sources*

The repository holds an annotated bibliography to support later papers. After it was drafted, a second review pass re-checked all entries against Crossref and re-read the abstracts behind each claim. Of 77 entries, 76 were confirmed against Crossref and one against DataCite; none had an unverifiable bibliographic record. The pass nevertheless made 10 corrections to the BibTeX file and corrected or tightened 16 claims in the prose. Examples include a meta-analysis described as showing a general advantage when its abstract places the value of XR training where traditional training is impossible (Kaplan et al., 2021); a construction-safety study described as showing better hazard identification when the available abstract supports only trade-specific advantages; a small pilot study annotated with statements ("single-centre", "authors note small sample") that its abstract does not contain; and an author list written as "and others". Ten entries had no abstract in any index, so their findings could not be checked; those entries say so and claim no findings.

This pattern is consistent with the general concern that generative models produce fluent text that is not supported by the source (Ji et al., 2023). It is also a reason for the standing rule recorded for agents: do not invent citations, statistics or facts about companies or products. The additional references used in this manuscript were verified in the same way, against Crossref records and, for the three on AI-assisted programming, against abstracts retrieved from OpenAlex on November 8, 2026. A bibliographic record check confirms that a paper exists. It does not confirm that a sentence about it is right; only reading does that.

*Lesson.* Verification of references has two layers, record and claim, and an assistant is more reliable at the first. The control adopted was a second pass, with a written list of what was corrected.

## *5.6 The repository is the assistant's memory*

The work spanned many assistant sessions, and sessions end: context is summarised or lost. Continuity was carried by files, not by the assistant. The instructions file, task list, status narrative, test record and a small set of saved notes about the owner's preferences were what a new session read first. A concrete example: when a generated Word document's images did not fit, the owner asked for the images as an HTML page, and later said not to make Word documents at all; the rule was saved so that it applies to future sessions, and this manuscript is accordingly an HTML page. Another: a decision to commit a small, shrunk subset of the

Rocketbox avatars (18 MB) for one project was recorded in the project's notes so that it would not be re-argued.

*Lesson.* With an assistant, documentation is operational, not decorative. The cost is that the documents themselves need the same care as code. The documentation checker that compares quoted test counts with a fresh run caught stale numbers, and a brand-kit test that had been failing since a November 5 commit was found by it on November 8 (the test's text-drawing rule did not anticipate a new toolbox file).

## *5.7 Parallel agents can diverge*

A branch named for OpenXR settings and VR hand models existed with one commit dated November 4, from a separate agent working on the same multi-user labs. It added an in-headset toolbox, hand models, a server “clear” message, a floor-level AR session mode and an AR shortcut, plus regenerated Unity OpenXR settings. The main branch received its own toolbox and hands the next day, apparently written without reference to the branch. Reviewing the two for the owner meant reading both, running the relay tests on each, loading the branch's page in an emulated Quest, and comparing designs: the main version had the fuller panel (examples, rotate, delete, open and close, reset) and 41 automated checks; the Unity changes were what the editor writes on opening a project and were already unnecessary for working player builds. One idea was worth keeping, a solo “press grip to drop the board on the floor” shortcut for AR alignment, and it was ported to main and tested in the emulator; the branch was then deleted with its last commit recorded.

*Lesson.* Ownership rules inside one session (Section 3.2) did not cover work started from a different session. Two implementations of one feature cost review time for a human who then had to decide between them. The practical control is to record who owns each area in the task list before any second agent starts, and to review stray branches before the final week.

## *5.8 Proxy measures and the realism investment*

The last week included heavy effort on lighting, sound and people. Performance was tracked with proxies: in the editor, SetPass-call counts at each scene's start view. For the hub's warehouse, the count fell from a round-1 mean of 3,196 to a median of 1,301 (mean 1,415) after capping pixel lights and other measures. These are CPU-side render counters in the editor. They are not a frame rate and say nothing about GPU time at headset resolution. Sound was rebuilt and measured for loudness but never heard in a scene; the lobby music, a synthesised original loop, was confirmed only to generate without errors. Similarly, whether the lighting looks good in a headset is a judgment no tool here could make.

There is also a tension with the literature in Section 2.1. Evidence that immersive realism can raise presence while lowering learning (Makransky et al., 2019; Parong & Mayer, 2018; Poupard et al., 2025) argues for testing the realism work against simpler versions before treating it as an improvement. The repository's study notes list a low-versus-high realism comparison for that reason, but the build effort ran ahead of any such evidence. An assistant makes polishing cheap, so the discipline of asking whether polish serves learning must come from the educator.

*Lesson.* Cheap production changes what is scarce. In this project, assistant effort was abundant, while human attention on a headset, evidence about learning and review capacity were scarce. Work should be sequenced toward the scarce resources: hardware time first, learning evidence next, polish last.

# 6. Discussion

## *6.1 What worked*

The arrangement delivered breadth that one educator could not have produced alone in three weeks: a working set of scenarios on two platform families, with documentation, automated checks and build tooling. The practices that appear to have mattered most were not clever prompting but ordinary engineering discipline written down: an instructions file, a task list, explicit ownership, review rounds, automated guards created from each failure, and honest labels on what was verified. The most valuable single rule was the owner's: never state that hardware works unless it was tested there.

## *6.2 How the observations relate to studies of AI programming assistants*

Vaithilingam et al. (2022) report that users struggled to understand, edit and debug generated code. Here the owner mostly directed and reviewed, and a large share of the debugging was done by the assistant, so the difficulty took a different form: the owner could not easily judge from the repository alone what a given claim rested on. The ladder and the verification notes are a response to that. Barke et al. (2023) distinguish acceleration and exploration. The routine, well-specified chores in this project (scripts, checks, documentation, porting a pattern across four Unity projects) look like acceleration. The exploratory parts (what the hub lobby should feel like, whether a toolbox should float or attach to the wrist) required the owner's judgment and, ultimately, a hardware test. This reading is an interpretation of one case, not a test of that theory.

### *6.3 Recommendations for educators*

1. **State an evidence ladder at the start** and label every claim and every screenshot (for example “emulated Quest 3”, “editor render”).
2. **Book hardware time early.** The scarcest resource was a headset session; features that depend on stereo, comfort or controllers should be scheduled against it from the first week, not the last.
3. **Build and run the player** before any handover, and treat log lines as test results.
4. **Turn every integration failure into a guard** (a script, a check, a sentence in a setup guide).
5. **Verify references in two layers**: the record, then the claim. Do not claim findings for sources whose abstracts could not be read.
6. **Treat the repository as the assistant's memory**: instructions file, task list, status narrative, test record. Review them as carefully as code.
7. **Assign ownership of each area before starting a second agent,** and review branches regularly.
8. **Sequence polish after evidence.** Test whether realism helps learning in the target task before investing further.
9. **Keep licences in the design.** Decide early what may be committed, and make the unbuildable-on-a-fresh-clone cases explicit in tooling and guides.

### *6.4 Threats to validity*

*Single case, single developer*. The findings may not transfer to larger teams or other assistants. *Self-report and authorship*. The author is both director and subject, and the assistant wrote much of the record that the findings rest on, so incidents that went unrecorded are missing. *Attribution.* Commits cannot separate human and assistant effort. *Timing*. The account ends on the day of the first planned hardware test; the lab results may change the proportions reported here. *No outcome data.* Nothing in this report speaks to whether the examples teach anything.

## 7. Limitations and future work

As of November 8, 2026, no example had been tested on a real Vive or Quest. All “VR” pictures in the repository are labelled as emulated or as editor renders. No learning outcome, comfort measure or usability measure has been collected. The Unity audio had not been heard in a scene. The biology scenario is labelled as a draft for faculty review. A section recording the lab-test results should be added when they exist.

The repository's study notes outline three comparisons that the examples are built to support: hands-on tasks versus a choose-a-response format; low versus high visual realism for the same scenario; and a platform comparison across the Vive, the Quest and a desktop fallback. Any such study should use retention and transfer measures, which the reviews in Section 2.1 identify as under-measured, and standard instruments such as the Simulator Sickness Questionnaire (Kennedy et al., 1993) and a presence questionnaire (Schubert et al., 2001). Any study with human participants requires review by the institution's human-subjects board before data are collected. A natural follow-up to this report is a retrospective comparison with a second project built without an assistant, or with a different division of labour, using the same ladder.

# 8. Conclusion

In three weeks, one educator and an AI coding assistant produced a large set of XR training examples, along with the tests, documents and build tooling to support them. The main problems were not in producing code. They were in knowing what the code did on devices that were not in the room, in keeping claims proportionate to evidence, and in keeping parallel work and external limits from colliding. The practices that addressed them were mostly low-tech: write down the rules, label the evidence, automate each lesson, and keep a human in charge of what counts as verified. Whether the examples help anyone learn remains to be tested.

# Declarations

**Use of AI.** Most of the code, documentation and records described here were produced in sessions with Claude (Anthropic), used through Claude Code; the trailers of the most recent commits name Claude Sonnet 5.5, and the model used in earlier sessions is not recorded in the repository. The text of this manuscript was drafted by Claude from the repository record, with references checked against Crossref and OpenAlex as described in Section 5.5. The author directed the work and is responsible for the content. *[Author to review the draft and confirm this statement against the target venue's policy before submission.]*

**Ethics.** No human participants took part and no personal data were collected. Future studies will require institutional review.

**Data and materials availability.** The repository is public at github.com/jpcst52/ALL-VR-MR-XR-SDKs. Licensed Asset Store packs and the Unity hub's scenes are not included, for licence reasons (see `docs/BUILD-THE-HUB.md`). The Microsoft Rocketbox avatars are MIT licensed. Brand assets are excluded except where publication was approved.

**Funding and competing interests.** *[Author to complete.]*

## References

All entries were checked against Crossref records (title, authors, year, venue, volume, pages, DOI). Entries marked † were also checked against an abstract. Author initials follow Crossref where it lists no given names in full.


1. Barke, S., James, M., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages, 7*(OOPSLA1), 85–111. https://doi.org/10.1145/3586030 †
2. Buttussi, F., & Chittaro, L. (2018). Effects of different types of virtual reality display on presence and learning in a safety training scenario. *IEEE Transactions on Visualization and Computer Graphics, 24*(2), 1063–1076. https://doi.org/10.1109/TVCG.2017.2653117 †
3. Dunmoye, I. D., Smith, L. T., Brown, J. S., Martin, J. P., May, D., & Hunsu, N. (2025). A systematic review on the use of collaborative virtual reality in engineering education. *Virtual Reality, 30*(1), Article 31. https://doi.org/10.1007/s10055-025-01280-4 †
4. Gonzalez-Franco, M., Ofek, E., Pan, Y., Antley, A., Steed, A., Spanlang, B., Maselli, A., Banakou, D., Pelechano, N., Orts-Escolano, S., Orvalho, V., Trutoiu, L., Wojcik, M., Sanchez-Vives, M. V., Bailenson, J., Slater, M., & Lanier, J. (2020). The Rocketbox library and the utility of freely available rigged avatars. *Frontiers in Virtual Reality, 1*, Article 561558. https://doi.org/10.3389/frvir.2020.561558
5. Hamilton, D., McKechnie, J., Edgerton, E., & Wilson, C. (2021). Immersive virtual reality as a pedagogical tool in education: A systematic literature review of quantitative learning outcomes and experimental design. *Journal of Computers in Education, 8*(1), 1–32. https://doi.org/10.1007/s40692-020-00169-2 †
6. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625 (page range as recorded by Crossref)
7. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730 †
8. Johnson-Glenberg, M. C. (2018). Immersive VR and education: Embodied design principles that include gesture and hand controls. *Frontiers in Robotics and AI, 5*, Article 81. https://doi.org/10.3389/frobt.2018.00081

9. Kaplan, A. D., Cruit, J., Endsley, M., Beers, S. M., Sawyer, B. D., & Hancock, P. A. (2021). The effects of virtual reality, augmented reality, and mixed reality as training enhancement methods: A meta-analysis. *Human Factors, 63*(4), 706–726. https://doi.org/10.1177/0018720820904229 †
10. Kennedy, R. S., Lane, N. E., Berbaum, K. S., & Lilienthal, M. G. (1993). Simulator Sickness Questionnaire: An enhanced method for quantifying simulator sickness. *International Journal of Aviation Psychology, 3*(3), 203–220. https://doi.org/10.1207/s15327108ijap0303_3
11. Khundam, C., Sukkriang, N., & Noël, F. (2021). No difference in learning outcomes and usability between using controllers and hand tracking during a virtual reality endotracheal intubation training for medical students in Thailand. *Journal of Educational Evaluation for Health Professions, 18*, Article 22. https://doi.org/10.3352/jeehp.2021.18.22 †
12. Makransky, G., Terkildsen, T. S., & Mayer, R. E. (2019). Adding immersive virtual reality to a science lab simulation causes more presence but less learning. *Learning and Instruction, 60*, 225–236. https://doi.org/10.1016/j.learninstruc.2017.12.007 †
13. Mayer, R. E., & Moreno, R. (2003). Nine ways to reduce cognitive load in multimedia learning. *Educational Psychologist, 38*(1), 43–52. https://doi.org/10.1207/s15326985ep3801_6
14. Parong, J., & Mayer, R. E. (2018). Learning science in immersive virtual reality. *Journal of Educational Psychology, 110*(6), 785–797. https://doi.org/10.1037/edu0000241 †
15. Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302
16. Poupard, M., Larrue, F., Sauzéon, H., & Tricot, A. (2025). A systematic review of immersive technologies for education: Learning performance, cognitive load and intrinsic motivation. *British Journal of Educational Technology, 56*(1), 5–41. https://doi.org/10.1111/bjet.13503 †
17. Poyade, M., Eaglesham, C., Trench, J., & Reid, M. (2021). A transferable psychological evaluation of virtual reality applied to safety training in chemical manufacturing. *ACS Chemical Health & Safety, 28*(1), 55–65. https://doi.org/10.1021/acs.chas.0c00105 †
18. Radianti, J., Majchrzak, T. A., Fromm, J., & Wohlgenannt, I. (2020). A systematic review of immersive virtual reality applications for higher education: Design elements, lessons learned, and research agenda. *Computers & Education, 147*, Article 103778. https://doi.org/10.1016/j.compedu.2019.103778 †

19. Rocha Estrada, F. J., Ruiz-Ramírez, J. A., George-Reyes, C. E., & Glasserman-Morales, L. D. (2022). Evaluation of a virtual campus adapted to web-based virtual reality spaces: Assessments of teachers and students. *Frontiers in Education, 7*, Article 918125. https://doi.org/10.3389/feduc.2022.918125 †

20. Saran, M. (2025). Comparing hand-based and controller-based interactions in virtual reality learning: Effects on presence and interaction performance. *PeerJ Computer Science, 11*, Article e3168. https://doi.org/10.7717/peerj-cs.3168 †

21. Schubert, T., Friedmann, F., & Regenbrecht, H. (2001). The experience of presence: Factor analytic insights. *Presence: Teleoperators and Virtual Environments, 10*(3), 266–281. https://doi.org/10.1162/105474601300343603

22. Scorgie, D., Feng, Z., Paes, D., Parisi, F., Yiu, T. W., & Lovreglio, R. (2024). Virtual reality for safety training: A systematic literature review and meta-analysis. *Safety Science, 171*, Article 106372. https://doi.org/10.1016/j.ssci.2023.106372 †

23. Slater, M. (2009). Place illusion and plausibility can lead to realistic behaviour in immersive virtual environments. *Philosophical Transactions of the Royal Society B: Biological Sciences, 364*(1535), 3549–3557. https://doi.org/10.1098/rstb.2009.0138 †

24. Stefan, H., Mortimer, M., & Horan, B. (2023). Evaluating the effectiveness of virtual reality for safety-relevant training: A systematic review. *Virtual Reality, 27*(4), 2839–2869. https://doi.org/10.1007/s10055-023-00843-7 †

25. Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science, 12*(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4

26. Vaithilingam, P., Zhang, T., & Glassman, E. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts* (pp. 1–7). ACM. https://doi.org/10.1145/3491101.3519665 †

27. van der Meer, N., van der Werf, V., Brinkman, W.-P., & Specht, M. (2023). Virtual reality and collaborative learning: A systematic literature review. *Frontiers in Virtual Reality, 4*, Article 1159905. https://doi.org/10.3389/frvir.2023.1159905 †

# Appendix A. Where each number in this report comes from

| Number | Source |
|---|---|
| 63 commits, Oct 17 to Nov 8, 2026; per-day counts; 58 / 4 / 1 by author | `git log` and `git rev-list --count HEAD` at commit f4e5899 |
| 3,664 tracked files; about 20,600 lines of JS/MJS/CJS; about 29,900 lines of C#; about 113,000 words of Markdown | `git ls-files` filtered (vendor, generated site, builds excluded), counted with `wc`; approximate, and includes tooling scripts |
| 12 to 17 scenarios | `docs/PROJECT-STATUS.md` (12) and commit messages of Nov 1 (16) and Nov 3 (17) |
| Suite counts (36, 16, 64, 48, 81, 12, 4 of 4, 26) | `docs/TESTING.md` (recorded Nov 2–3); the relay (64) and brand-kit (16) suites and the 41-check toolbox suite were re-run on Nov 8 |
| 77 entries; 76 Crossref, 1 DataCite; 10 .bib corrections; 16 prose corrections; 10 entries without abstracts | `docs/research/VERIFICATION.md` |
| SetPass calls 3,196 to 1,301 (warehouse, hub) | `vive-steamvr/hub/README.md`, editor Play mode, Game view, Nov 3 (a CPU-side counter, not frame rate) |
| Launcher images 3.3 to 0.84 MB and 3.0 to 0.80 MB; 3 MB limit | File sizes before and after; `scripts/build-webxr.mjs` and `scripts/check-image-sizes.mjs` |
| Hub player: about 1 GB, started 45 s windowed; warehouse player 25 s windowless | Build output and run logs on the development PC, Nov 8; not independently reproduced |

# Appendix B. Incident log

| # | Date | Incident | Class | Control added |
|---|---|---|---|---|
| 1 | Not dated | Statement that a library "works on the Vive" rested only on a README; owner corrected it | Overclaim | Standing honesty rule; evidence ladder; labels on emulated pictures |
| 2 | Nov 3 | Research summaries needed correction or tightening in 16 places (several went beyond abstracts); 10 BibTeX fixes | Overclaim | Second verification pass; no findings for entries without abstracts |
| 3 | By Nov 5 | Text drawn in one eye only (shader lacked stereo-instancing macros) | Hardware-only defect | Stereo-aware shaders; desktop self-tests; headset checklist |
| 4 | Nov 4–5 | A second agent built a toolbox and hands on a separate branch while main got its own | Divergent parallel work | Branch review; one idea ported; branch deleted; ownership to be recorded before parallel starts |
| 5 | Nov 5–8 | A brand-kit test failed silently after a new file broke its text-drawing rule | Test rot | Rule refined; docs checker compares counts to a fresh run |
| 6 | Nov 8 | All four Unity player builds failed on an editor-only property in runtime code | Editor/player divergence | Build and run the player before handover |
| 7 | Nov 8 | Sky shader stripped from the hub player (`Shader.Find`); a first re-run under `-nographics` still showed the warning | Editor/player divergence; invalid check | Pre-build hook; verification with graphics on |
| 8 | Nov 8 | Pages deployment failed on a 3.3 MB screenshot (silent 3 MB cap) | Hidden platform limit | Images shrunk; `check-image-sizes` in the workflow |
| 9 | Nov 8 | Hub unbuildable from a fresh clone; hub-only build overwrote the shareable combined archive | Licence and tooling | Prerequisite check; hub excluded from combined archive; pack and unpack scripts; guide |
| 10 | Nov 8 | A Word report's images did not fit; a stuck editor from stale backup-scene data | Format and editor state | HTML-only rule saved to memory; stale backup data cleared and documented |